

The Programmatic Large-Scale Minecraft Terrain Architecture

A Technical Blueprint for Building Automated 10,000 × 10,000 Block Continents

Reference Implementation & Mathematical Specification · Minecraft 1.21.1+ · World Machine / Gaea to WorldPainter

1. Executive Summary & The Paradigm Shift

Creating massive, photorealistic Minecraft worlds at continental scale ($10\{, \}000 \times 10\{, \}000$ blocks, covering 100 km^2 and $100\{, \}000\{, \}000$ surface columns) cannot be accomplished through traditional manual brush painting in WorldPainter or in-game voxel tools like VoxelSniper and WorldEdit. Manual sculpting at this scale inevitably results in:

- Unnatural Repetition & Tool Artifacts:** Concentric brush rings, unnatural plateaus, flat-topped mountains, and repetitive height distributions.
- Broken Hydrology:** Disconnected river segments that pool into unnatural depressions or flow uphill.
- Razor-Sharp Ecological Seams:** Artificial, hard-line biome boundaries rather than continuous, slope-aware climatic transitions.
- Human Bottlenecks:** Hundreds of hours of manual labor that must be discarded and redone whenever continent geometry, sea level, or height limits change.

The professional workflow highlighted in the video replaces manual painting with an **automated, deterministic, multi-stage programmatic pipeline**:

STAGE 1: GEOLOGICAL SIMULATION (World Machine / Gaea /

- Spline-guided instance scattering for structural mountains
- Fluvial flow restructuring & monotonic downhill river carving
- Thermal weathering simulation (critical angle of repose)

Outputs: 16-bit Lossless Heightfield (10,000 × 10,000)



STAGE 2: ECOLOGICAL PROBABILITY & DISTRIBUTION TRANSFORMATION

- Continuous Environmental Multi-Criteria Evaluation (terrain, elevation, aspect)
- Mathematical response curves (treeline cutoff, slope-dependent vegetation)
- Dithering & Variable Poisson-Disk Sampling: continuous to discrete

Outputs: 8-bit Distribution Masks (Populate, Forest Type, etc.)



STAGE 3: HEADLESS WORLDPainter AUTOMATION ENGINE (JSR-200)

- Config-driven declarative compiler (`terrain\_config.json`)
- Automated Rhino ECMAScript generation (zero manual G-code)
- Subprocess headless execution via `wpscript` CLI
- Direct Anvil Region (.mca) chunk synthesis & Still Life generation

Outputs: Native Minecraft World Save (.minecraft/saves/)

2. Stage 1: Procedural Geological Synthesis

2.1 Instance Scattering for Realistic Mountain Ridgelines

Standard gradient noise (Perlin, Simplex) produces isotropic, amorphous mounds. Real tectonic mountain ranges consist of **structural cordilleras**, **knife-edge arêtes**, and **pyramidal peaks** formed along fault lines.

A. Mathematical Model

Instead of evaluating pure mathematical noise across a 2D plane, ridgelines are constructed by **scattering geometric instances along parametric guide curves**:

1. **Structural Fault Spline $\mathbf{C}(t)$** : A parametric cubic Bézier or Hermite curve defining the continental spine: $\mathbf{C}(t) = (x(t), z(t))$, $\quad t \in [0, 1]$ Tangent vector: $\mathbf{T}(t) = \frac{\mathbf{C}'(t)}{\|\mathbf{C}'(t)\|}$, Normal vector: $\mathbf{N}(t) = (-T_z, T_x)$.
2. **Primitive Instance Geometry**: Each scattered mountain primitive has local coordinate axes (u, v) aligned to $(\mathbf{T}, \mathbf{N})$:
 - **Arête / Ridge Primitive**: $h_{\text{ridge}}(u, v) = H_0 \cdot \exp\left(-\frac{|v|}{\sigma_{\text{width}}}\right) \cdot \left(1 - \left(\frac{u}{L}\right)^2\right) \cdot \left(1 + \eta \cdot \text{Noise}(u, v)\right)$
 - **Pyramidal Peak Primitive (Matterhorn-style cirque headwall)**: $h_{\text{peak}}(r, \theta) = H_{\text{peak}} \cdot \left(1 - \frac{r}{R}\right)^\gamma \cdot \left[1 + \alpha \cos(k \theta)\right]$ where k represents the number of radiating arêtes (typically 3 or 4) and $\gamma \approx 1.8$ creates concave glacially carved cirque flanks.
3. **Instance Scattering Algorithm**:
 - Sample points along $\mathbf{C}(t)$ using **Poisson-disk spacing** with spatial jittering.
 - At each sample point, instantiate a peak or ridge primitive with randomized height $H \sim \mathcal{N}(\mu_H, \sigma_H)$, width σ , and local orientation aligned to $\mathbf{T}(t) + \delta\theta$.

4. **Composite Blending via Smooth-Max:** Standard addition ($\sum h_i$) causes unnatural stacking and flat plateau tops. Standard maximum ($\max(h_i)$) leaves razor-sharp derivative discontinuities. The professional workflow uses **Polynomial Smooth-Max (smax)**: $\text{smax}(a, b, k) = \frac{a + b + \sqrt{(a - b)^2 + k}}{2}$ where k controls the blending radius, producing sharp crests with smooth saddle passes (cols).
-

2.2 Thermal Weathering & Talus Slumping

Rock walls cannot sustain vertical cliffs indefinitely; gravity and freeze-thaw cycles cause rock to fracture and accumulate at the base of slopes at the **critical angle of repose** ($\theta_c \approx 35^\circ - 40^\circ$).

A. Finite-Difference Numerical Implementation

Given a heightfield $H(x, z)$ on a grid with cell spacing Δx :

1. Calculate local slope angle $\theta(x, z) = \arctan(|\nabla H|)$.
 2. For every cell where $\theta(x, z) > \theta_c$, calculate excess slope: $\Delta h_{\text{excess}} = \max(0, |\nabla H| - \tan(\theta_c))$
 3. Displace mass downhill along the direction of steepest descent: $\mathbf{v}_{\text{flow}} = -\frac{\nabla H}{|\nabla H|}$
 4. Material eroded from the steep scarp is deposited in the concave foot zone, forming **characteristic parabolic talus scree aprons**: $H_{\text{talus}}(d) = H_{\text{base}} + d \cdot \tan(\theta_c - \beta) \cdot d^2$ where d is distance from the cliff face.
-

2.3 Flow Restructuring & Hydrological River Routing

In standard noise terrain, water collects in unnatural depressions (pits) and cannot reach the ocean. The video creator utilizes **flow restructuring** to force rivers along realistic, continuous downhill trajectories.

A. The 4-Step Hydrological Engine:

1. Pit Filling / Priority-Flood (Barnes / Wang-Liu Algorithm):

- Implements a priority queue to simulate a rising flood inward from the ocean boundary.
- Any landlocked depression is either raised to its lowest spillway elevation or carved with a continuous descending breach channel, guaranteeing that: $\forall i, \quad H(p_{i+1}) \leq H(p_i)$

2. D-Infinity Drainage Accumulation:

- Calculates water flow proportion across grid cells based on slope aspect $\alpha = \arctan2(-\frac{\partial H}{\partial x}, \frac{\partial H}{\partial z})$.
- Upstream catchment area $A(x, z)$ accumulates recursively: $A(p) = 1 + \sum_{q \in \text{inflow}(p)} w(q, p) \cdot A(q)$

3. Stream Power Law Incision:

- High-accumulation valleys carve deeper beds according to the geological Stream Power Equation: $\Delta H_{\text{river}} = -K_{\text{erosion}} \cdot A(x, z)^m \cdot |\nabla H|^n$ where $m \approx 0.5$ (drainage area exponent) and $n \approx 1.0$ (slope exponent).

4. Channel Cross-Section Carving:

- Headwaters / Mountain torrents ($A < 500$): Narrow V-shaped ravines.
- Mid-tier rivers ($500 \leq A < 5000$): U-shaped braided gravel channels.
- Continental lowland rivers ($A \geq 5000$): Wide, flat, alluvial meandering floodplains with natural levees and sandbars.

3. Stage 2: Mapping & Distribution (Feature Probability Conversion)

3.1 The Problem: Continuous Fields vs. Discrete Minecraft Blocks

Terrain synthesis software produces continuous 32-bit floating-point arrays ($H \in [-64.0, 320.0]$, slope $\theta \in [0^\circ, 90^\circ]$, curvature $\kappa \in [-1, 1]$, moisture $M \in [0, 1]$). However, Minecraft requires **discrete integer block IDs** and **exact point coordinates** for objects (trees, boulders, grass tufts, snow layers).

3.2 Environmental Multi-Criteria Evaluation (MCE)

For every Minecraft feature (e.g., *Dense Oak Canopy*, *Glacial Erratic Boulder*, *Riverbed Scree*, *Snow Crust*), we construct an ecological suitability function: $P_{\text{feature}}(x, z) = f_{\text{height}}(H) \cdot f_{\text{slope}}(\theta) \cdot f_{\text{moisture}}(M) \cdot f_{\text{curvature}}(\kappa) \cdot f_{\text{exclusion}}(x, z)$

Exact Ecological Formulations:

1. Foliage / Still Life Canopies (Oak / Birch / Pine):

- **Slope Filter:** $f_{\text{slope}}(\theta) = \frac{1}{1 + \exp\left(0.4 \cdot (\theta - 32^\circ)\right)}$ (*1.0 below 25° , drops smoothly to 0 at $>35^\circ$*).
- **Treeline Filter:** $f_{\text{height}}(H) = \begin{cases} 1.0 & \text{if } H \leq 180 \cdot \frac{225 - H}{45} \\ 0.0 & \text{if } 180 < H \leq 225 \\ 0.0 & \text{if } H > 225 \end{cases}$
- **Water Buffer:** Excludes $H \leq \text{SeaLevel} + 1.0$ to prevent submerged trees.

2. Alpine Scree & Bare Bedrock:

- $P_{\text{rock}}(\theta) = \frac{1}{1 + \exp\left(-0.5 \cdot (\theta - 38^\circ)\right)}$ (*Dominates exclusively when slope exceeds 40°*).

3. Glacial Permafrost & Snow:

- $P_{\text{snow}}(H, \theta) = \text{clamp}\left(\frac{H - 200}{40}, 0, 1\right) \cdot (1 - 0.6 \cdot \sin\theta)$ (*Steep cliff faces shed snow; flat summits retain thick crust*).

3.3 Converting Continuous Probability to Pixelated Spatial Distributions

If a cell has suitability $P(x, z) = 0.35$, how does the software decide whether to spawn a tree at that specific pixel?

Method A: Blue Noise / Void-and-Cluster Dithering (Best for Surface Covers)

Standard white noise creates clumps and large voids. **Blue noise** distributes placement points with maximum spatial separation between neighbors: $\text{Mask}(x, z) = \begin{cases} 255 & \text{if } P(x, z) \geq \text{BlueNoiseMatrix}_{[64 \times 64]}(x \bmod 64, z \bmod 64) \setminus 0 \\ 0 & \text{otherwise} \end{cases}$ *Result:* A pixelated binary distribution map that looks completely natural to the human eye, with zero grid repetition or clumping.

Method B: Variable-Radius Poisson-Disk Sampling (Best for Custom Trees / Schematics)

For structural objects with physical bounding boxes (e.g., a $9 \times 9 \times 12$ block procedural oak tree):

1. Compute the local target density $\rho(x, z) = \rho_{\max} \cdot P_{\text{feature}}(x, z)$.
2. Calculate the required minimum spacing: $r(x, z) = \frac{r_{\min}}{\sqrt{\rho(x, z)}}$.
3. Run Bridgson's $O(N)$ Poisson-disk point-scattering across the 2D plane.
4. Output the resulting point coordinates as an exact custom object layer or export mask for WorldPainter.

4. Stage 3: Headless WorldPainter Automation (JSR-223 Scripting)

4.1 Why Scripting Replaces Manual Painting

In a \$10\{\,}000 \times 10\{\,}000\$ map, importing a 16-bit heightmap, configuring 8 distinct terrain strata, importing 6 layer masks, setting opacity thresholds, and configuring export parameters takes 45–60 minutes of repetitive mouse clicking.

WorldPainter exposes a complete **Java Scripting API (JSR-223)** powered by the Mozilla Rhino ECMAScript engine. A single automated script can execute the entire import and synthesis deterministically in seconds.

4.2 Declarative Pipeline Architecture

The system separates **world configuration** from **execution logic**:

```
# terrain_config.yaml
world:
  name: "Ashenfall_Continent"
  width: 10000
  height: 10000
  min_y: -64
  max_y: 320
  sea_level: 62

inputs:
  heightmap: "exports/height_16bit.png"
  populate_mask: "exports/mask_populate.png"
  frost_mask: "exports/mask_frost.png"
  scree_mask: "exports/mask_scree.png"

strata:
  - range: [-64, 61]
    terrain: "Gravel / Sand"          # Ocean basin & coastli
  - range: [62, 130]
    terrain: "Grass / Bare Dirt"     # Fertile valleys & pla
    slope_limit: [0, 35]
  - range: [131, 190]
    terrain: "Podzol / Coarse"      # Subalpine pine forest
  - range: [191, 250]
    terrain: "Stone / Cobblestone"# High crags & arêtes
```

```
- range: [251, 320]
  terrain: "Deep Snow"           # Glacial summits
```

4.3 Headless JSR-223 Rhino Script Structure

The automated JavaScript executed by `wpscript`:

```
// ashenfall_automated_synthesis.js
print("[1/4] Loading 16-bit Master Heightfield...");
var heightMap = wp.getHeightMap()
    .fromFile("exports/height_16bit.png")
    .go();

print("[2/4] Sculpting 3D Dimension Geometry...");
var world = wp.createWorld()
    .fromHeightMap(heightMap)
    .scale(100)
    .fromLevels(0, 65535).toLevels(-64, 320)
    .withWaterLevel(62)
    .withLowerBuildLimit(-64)
    .withUpperBuildLimit(320)
    .go();

print("[3/4] Applying Programmatic Distribution Masks...")
// Populate Mask (Still Life Vegetation)
var popMask = wp.getHeightMap().fromFile("exports/mask_po
var popLayer = wp.getLayer().withName("Populate").go();
wp.applyHeightMap(popMask).toWorld(world).applyToLayer(po

// Glacial Frost Layer
var frostMask = wp.getHeightMap().fromFile("exports/mask_
var frostLayer = wp.getLayer().withName("Frost").go();
wp.applyHeightMap(frostMask).toWorld(world).applyToLayer(

print("[4/4] Exporting Final Minecraft Region Files (.mca
wp.exportWorld(world)
    .toDirectory(".minecraft/saves/Ashenfall")
    .withGameVersion("1.19 or later (Deepslate / 384 bloc
```

```
.go();  
print("[✓] Turnkey World Synthesis Complete!");
```

5. Master Software Architecture: How to Build the Program

To build a standalone, unified application that performs this complete pipeline from scratch, the system is architected as **five modular decoupled engines**:

```
ashenfall_engine/  
├── core/  
│   ├── instance_scatterer.py    # Spline evaluation, Poisson  
│   ├── thermal_weathering.py    # Mass-conserving cellular  
│   ├── hydraulic_router.py      # Priority-Flood pit fill  
│   └── color_quantizer.py       # creativity Euclidean R  
├── distribution/  
│   ├── ecological_evaluator.py  # Continuous multi-criter  
│   ├── blue_noise_dither.py     # Void-and-Cluster contin  
│   └── mask_exporter.py         # 8-bit / 16-bit PNG asse  
├── automation/  
│   ├── config_parser.py        # Validates YAML/JSON ter  
│   ├── jsr223_compiler.py      # Emits deterministic Wor  
│   └── wpscript_bridge.py       # Subprocess runner findi  
└── cli.py                      # Turnkey master CLI runn
```

5.1 Module 1: Spline-Guided Mountain Ridgeline Scatterer (`instance_scatterer.py`)

```
import numpy as np  
from scipy.interpolate import CubicSpline  
from typing import List, Tuple
```

```

class RidgeScatterEngine:
    def __init__(self, res: int = 4096):
        self.res = res
        self.grid = np.zeros((res, res), dtype=np.float32)

    def scatter_along_spline(
        self,
        waypoints: List[Tuple[float, float]],
        peak_count: int = 40,
        base_altitude: float = 240.0,
        ridge_sigma: float = 18.0
    ):
        """
        Scatters arête and peak instances along a structure
        """
        waypoints = np.array(waypoints)
        t = np.linspace(0, 1, len(waypoints))
        cs_x = CubicSpline(t, waypoints[:, 0])
        cs_z = CubicSpline(t, waypoints[:, 1])

        # Sample uniform stations along spline
        t_samples = np.linspace(0, 1, peak_count)
        for ts in t_samples:
            cx, cz = float(cs_x(ts)), float(cs_z(ts))
            dx, dz = float(cs_x(ts, 1)), float(cs_z(ts, 1))
            tangent_angle = np.arctan2(dz, dx)

            # Jittered peak attributes
            h_peak = base_altitude * np.random.uniform(0.5, 1.0)
            length = np.random.uniform(120, 260)

            self._render_arete_instance(cx, cz, tangent_angle, h_peak, length)

    def _render_arete_instance(self, cx, cz, angle, height, length):
        # Local bounding box calculation
        rad = int(max(length, sigma * 4))
        x_min = max(0, int(cx - rad))
        x_max = min(self.res, int(cx + rad))
        z_min = max(0, int(cz - rad))

```

```

z_max = min(self.res, int(cz + rad))

X, Z = np.meshgrid(np.arange(x_min, x_max), np.arange(z_min, z_max))
dx = X - cx
dz = Z - cz

# Rotate into ridge coordinate frame (u = along ridge)
cos_a, sin_a = np.cos(angle), np.sin(angle)
u = dx * cos_a + dz * sin_a
v = -dx * sin_a + dz * cos_a

# Sharp exponential cross-section (arête) + quadratic falloff
along_falloff = np.clip(1.0 - (u / (length / 2.0)), 0.0, 1.0)
cross_profile = np.exp(-np.abs(v) / sigma)
instance_h = height * along_falloff * cross_profile

# Smooth-Max compositing (avoids plateau truncation)
sub_grid = self.grid[z_min:z_max, x_min:x_max]
k = 12.0 # Smoothness factor
self.grid[z_min:z_max, x_min:x_max] = 0.5 * (sub_grid + k * np.sqrt(1 + (sub_grid - instance_h)**2))

```

5.2 Module 2: Mass-Conserving Thermal Weathering & Fluvial Carver (`thermal_weathering.py`)

```

import numpy as np

def apply_thermal_weathering(
    heightfield: np.ndarray,
    critical_angle_deg: float = 38.0,
    iterations: int = 15,
    talus_transfer_rate: float = 0.35
) -> np.ndarray:
    """
    Simulates freeze-thaw rockfall and talus scree fan deposition.
    """
    H = heightfield.copy()
    crit_slope = np.tan(np.radians(critical_angle_deg))

```

```

ny, nx = H.shape

for _ in range(iterations):
    # Calculate 4-directional slopes
    dh_dx_pos = H[:, 1:] - H[:, :-1]
    dh_dy_pos = H[1:, :] - H[:-1, :]

    # Downhill divergence tracking
    eroded = np.zeros_like(H)

    # West-to-East
    excess_x = np.maximum(0.0, dh_dx_pos - crit_slope)
    transfer_x = excess_x * talus_transfer_rate
    eroded[:, :-1] += transfer_x
    eroded[:, 1:] -= transfer_x

    # North-to-South
    excess_y = np.maximum(0.0, dh_dy_pos - crit_slope)
    transfer_y = excess_y * talus_transfer_rate
    eroded[:-1, :] += transfer_y
    eroded[1:, :] -= transfer_y

    H -= eroded
return H

```

5.3 Module 3: Continuous-to-Discrete Blue Noise Ditherer (blue_noise_dither.py)

```

import numpy as np
from PIL import Image

def generate_void_and_cluster_blue_noise(size: int = 64)
    """
    Generates a deterministic 64x64 blue noise threshold
    """
    np.random.seed(42)
    # Standard Bayer dither matrix approximation for demo

```

```

matrix = np.random.uniform(0.0, 1.0, (size, size))
# Gaussian bandpass filtering to isolate high spatial
from scipy.ndimage import gaussian_filter
matrix = matrix - gaussian_filter(matrix, sigma=1.5)
matrix = (matrix - matrix.min()) / (matrix.max() - ma
return matrix

def continuous_to_discrete_mask(
    probability_field: np.ndarray,
    threshold_matrix: np.ndarray
) -> np.ndarray:
    """
    Converts continuous suitability P(x, z) in [0, 1] into
    un-clumped binary placement mask for Minecraft object
    """
    ny, nx = probability_field.shape
    ty, tx = threshold_matrix.shape

    # Tile the dither pattern across the continental domain
    tiled_threshold = np.tile(threshold_matrix, (ny // ty

    # Binary pixelated decision
    discrete_mask = np.where(probability_field >= tiled_t
    return discrete_mask

```

6. Implementation Summary & Direct Application to This Repo

The reason the creator's video seems so impressive is that it synthesizes three distinct fields of computer science into a single pipeline:

1. **Applied Geomorphology:** Stream power laws, critical angles of repose, and tectonic spine curvature.
2. **Signal Processing & Spatial Statistics:** Converting continuous floating-point fields into blue-noise dithered stochastic point

patterns.

3. **Compiler & Automation Architecture:** Decoupling artistic intent into declarative config files and executing them headlessly via JSR-223.

In our repository:

- `tools/ashfall_engine/generator.py` executes **Stage 1** (Hermite spine, alpine ridgelines, and caldera subsidence).
- `tools/ashfall_engine/color_to_terrain.py` executes **Stage 2** (creativitRy Euclidean block quantization and slope-aware masks).
- `tools/worldpainter_api.py` and `generate_ashfall.py` execute **Stage 3** (automated JSR-223 Rhino script compilation and headless WorldPainter packaging).

This specification document provides the exact mathematical formulas, data structures, and algorithmic implementations needed to build and scale this software architecture to any custom terrain project.